

Naive Bayes

Classification via Bayes' theorem

Supervised Learning · Classification

$$P(\text{class} \mid \text{features}) = \frac{P(\text{features} \mid \text{class})P(\text{class})}{P(\text{features})}$$

generative probability · conditional independence · fast closed-form estimates

Bayes' Theorem for Classification

$$P(y = k | x) = \frac{P(x | y = k)P(y = k)}{P(x)} \propto P(x | y = k)P(y = k)$$

$$\hat{y} = \arg \max_k P(x | y = k)P(y = k)$$

Prior

$P(y=k)$

class frequency

Likelihood

$P(x | y=k)$

feature model

Posterior

$P(y=k | x)$

class after evidence

Evidence

$P(x)$

same for every k

Drop $P(x)$ only for the arg-max: it changes normalization, not the winning class.

Putting the Pieces Together

The probability that an input belongs to a class, given its features, comes from the probability of seeing those features in that class (the **feature model**) multiplied by the **prior probability** of that class.

$$P(\text{class} \mid \text{features}) = \frac{P(\text{features} \mid \text{class}) \times P(\text{class})}{P(\text{features})}$$

Feature model

$P(\text{features} \mid \text{class})$

How well do these features fit the class?

Prior

$P(\text{class})$

Class frequency before seeing the features.

Normalize

Divide by $P(\text{features})$

Makes class probabilities sum to 1.

To choose the most likely class, compare feature model $\times$ prior; the normalization term is the same for each class.

Feature Model

The feature model tells Naive Bayes how to calculate the probability of observing a feature given a particular class.

In probability notation: the likelihood of the features given the class.

$$P(\text{features} \mid \text{class})$$

Example: A Fruit

Suppose we describe each fruit with three features:

Weight

150 g

Length

8 cm

Round?

Yes

What is the probability this fruit is an apple, given its features?

$$P(\text{class} = \text{apple} \mid w = 150 \text{ g}, l = 8 \text{ cm}, r = \text{yes}) = \frac{P(w = 150 \text{ g}, l = 8 \text{ cm}, r = \text{yes} \mid \text{class} = \text{apple}) \times P(\text{class} = \text{apple})}{P(w = 150 \text{ g}, l = 8 \text{ cm}, r = \text{yes})}$$

The numerator combines the feature model and the prior; the denominator normalizes the result.

The Naive Assumption

If we assume that the class is apple, treat the three feature values as independent of one another.

Weight

considered on its own

Length

considered on its own

Round?

considered on its own

"Naive" means we ignore relationships between features after conditioning on the class.

Multiply the Feature Probabilities

With the naive assumption, estimate each likelihood separately, then multiply:

$$P(w = 150 \text{ g}, l = 8 \text{ cm}, r = \text{yes} \mid \text{apple}) \approx \\ P(w = 150 \text{ g} \mid \text{apple}) \times P(l = 8 \text{ cm} \mid \text{apple}) \times P(r = \text{yes} \mid \text{apple})$$

probability an apple weighs 150 g

probability an apple is 8 cm long

probability an apple is round

Why make this assumption? Weight and length may be related, so the product may only approximate the true joint probability. Treating features separately makes the model much easier to estimate.

Naive Bayes makes this simplifying assumption even when it is not perfectly true.

Fitting the Likelihood Model

Use the training data to estimate how each feature behaves **within each class**.

Continuous

Choose a distribution

Often Gaussian, or another suitable continuous probability distribution.

Fit it to training values

For a Gaussian, estimate the mean and variance for each feature in each class.

Binary

Use a Bernoulli model

For each class, estimate the fraction of its training examples where this feature is 1.

Counts

Use a Multinomial model

For each class, estimate word or event probabilities from their training counts. Smoothing gives unseen words a small nonzero probability.

The likelihood model is learned from training data; then it scores how likely a new input's features are under each class.

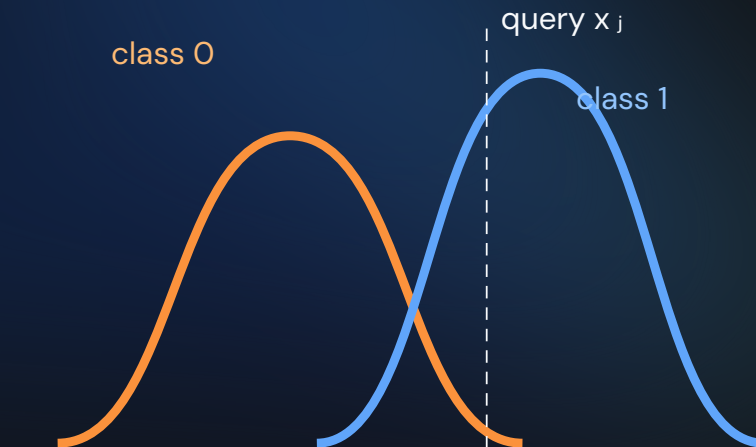
Gaussian Naive Bayes · Continuous Features

$$P(x_j | y = k) = \frac{1}{\sqrt{2\pi\sigma_{jk}^2}} \exp\left[-\frac{(x_j - \mu_{jk})^2}{2\sigma_{jk}^2}\right]$$

$$\hat{\mu}_{jk} = \frac{1}{n_k} \sum_{i:y_i=k} x_{ij}, \quad \hat{\sigma}_{jk}^2 = \frac{1}{n_k} \sum_{i:y_i=k} (x_{ij} - \hat{\mu}_{jk})^2$$

```
from sklearn.datasets import make_classification
from sklearn.naive_bayes import GaussianNB
```

```
X, y = make_classification(
    n_samples=200, n_features=4,
    n_informative=3, n_redundant=0,
    random_state=0)
model = GaussianNB().fit(X, y)
print(model.theta_, model.var_)
print(model.predict_proba(X[:3]))
```



Counts, Binary Features, and Smoothing

Multinomial NB

word or event counts

Bernoulli NB

presence / absence

$$P(x_j = v \mid y = k) = \frac{\text{count}(x_j = v, y = k) + \alpha}{\text{count}(y = k) + \alpha|V|}$$

Without smoothing: one unseen word gives probability 0, collapsing the entire product.

```
from sklearn.feature_extraction import text
from sklearn.naive_bayes import MultinomialNB

docs = ["free money click", "project meeting",
        "win free prize", "project deadline"]
y = [1, 0, 1, 0]
vectorizer = text.CountVectorizer()
X = vectorizer.fit_transform(docs)
model = MultinomialNB(alpha=1).fit(X, y)
new = vectorizer.transform(["free prize click"])
print(model.predict(new), model.predict_proba(new))
```

Naive Bayes vs. Logistic Regression

	Naive Bayes	Logistic Regression
Models	$P(x \mid y)$ then Bayes	$P(y \mid x)$ directly
Type	generative	discriminative
Fit	closed-form statistics	iterative optimization
Thrives on	small, sparse, high-d data	correlated predictive features
Risk	independence distorts probabilities	more data and fitting time

Under certain Gaussian assumptions, both converge to the same boundary with abundant data—but reach it differently and behave differently when data is scarce.

Naive Bayes in One View

Invert

likelihood $\times$ prior becomes posterior ranking

Factor

conditional independence makes the joint tractable

Match the variant

Gaussian, Multinomial, or Bernoulli

Smooth

avoid zero-probability collapse

Next: **Decision Trees** — interpretable if/else rules with no probability model or distance metric.

Spam Detection Setup

Classes: **spam** and **not spam**

free
word appears

prize
word appears

meeting
word appears

New message: **"free prize"**

Estimate a score for each class:

$$\text{score}(\text{class}) = P(\text{message} \mid \text{class}) \times P(\text{class})$$

The class with the larger score is the prediction. Normalize the scores if you want probabilities.

Spam Detection Recipe: Build Likelihoods

Extract the likelihoods from the training messages, separately for each class.

Multinomial NB

Count tokens

For each class, count how many times each word appears in all of its messages.

Divide by all tokens

$P(\text{free} \mid \text{spam}) = \frac{\text{free tokens in spam}}{\text{all spam tokens}}$

Repeated words count repeatedly.

Bernoulli NB

Count messages

For each class, count how many messages contain each word at least once.

Divide by messages

$P(\text{free} \mid \text{spam}) = \frac{\text{spam messages containing free}}{\text{all spam messages}}$

Repeated words still count as one present feature.

These class-specific feature probabilities are the likelihood pieces of Naive Bayes. Add smoothing when a count is zero.

Spam Detection Recipe: Extract Priors

The prior is the class probability before looking at a new message.

Spam

40 spam messages

$$P(\text{spam}) = 40 \div 100 = .40$$

Not spam

60 not-spam messages

$$P(\text{not spam}) = 60 \div 100 = .60$$

$$\text{prior}(\text{class}) = \text{number of training messages in the class} \div \text{total training messages}$$

The prior reflects class balance. It is estimated once from the training labels and does not depend on the new message's words.

Spam Detection Recipe: Combine and Classify

For the new message "free prize," multiply its likelihood by each class prior.

Spam score

$$P(\text{free} \mid \text{spam}) \times P(\text{prize} \mid \text{spam}) \times P(\text{spam})$$

$$.30 \times .20 \times .40 = .024$$

Not-spam score

$$P(\text{free} \mid \text{not spam}) \times P(\text{prize} \mid \text{not spam}) \times P(\text{not spam})$$

$$.05 \times .01 \times .60 = .0003$$

Normalize the scores:

$$P(\text{spam} \mid \text{message}) = .024 \div (.024 + .0003) \approx .988$$

$$P(\text{not spam} \mid \text{message}) \approx .012$$

The larger posterior wins: classify the message as **spam**.

Questions?

Naive Bayes: model the class, factor the features, then apply Bayes' theorem.

Next: Decision Trees